15. Design Meeting Scheduling System

Key Terms:

Term	Full Name	What it does
UTC	Coordinated Universal Time	Global time standard for storage
RRULE	Recurrence Rule (iCalendar)	Standard format for recurring events
DST	Daylight Saving Time	Seasonal time shift (complicates scheduling)
Free/Busy	Availability data	Shows when users are available/busy
Bitmap	Binary array	Efficient storage for availability slots
TIMESTAMPTZ	Timestamp with timezone	PostgreSQL type storing time + zone

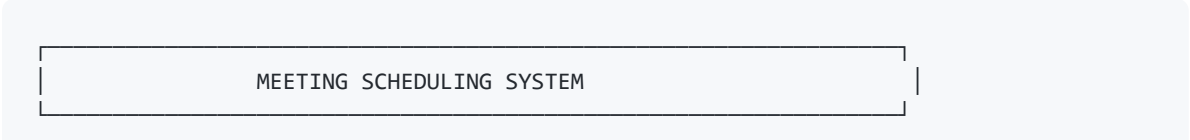
Problem Statement

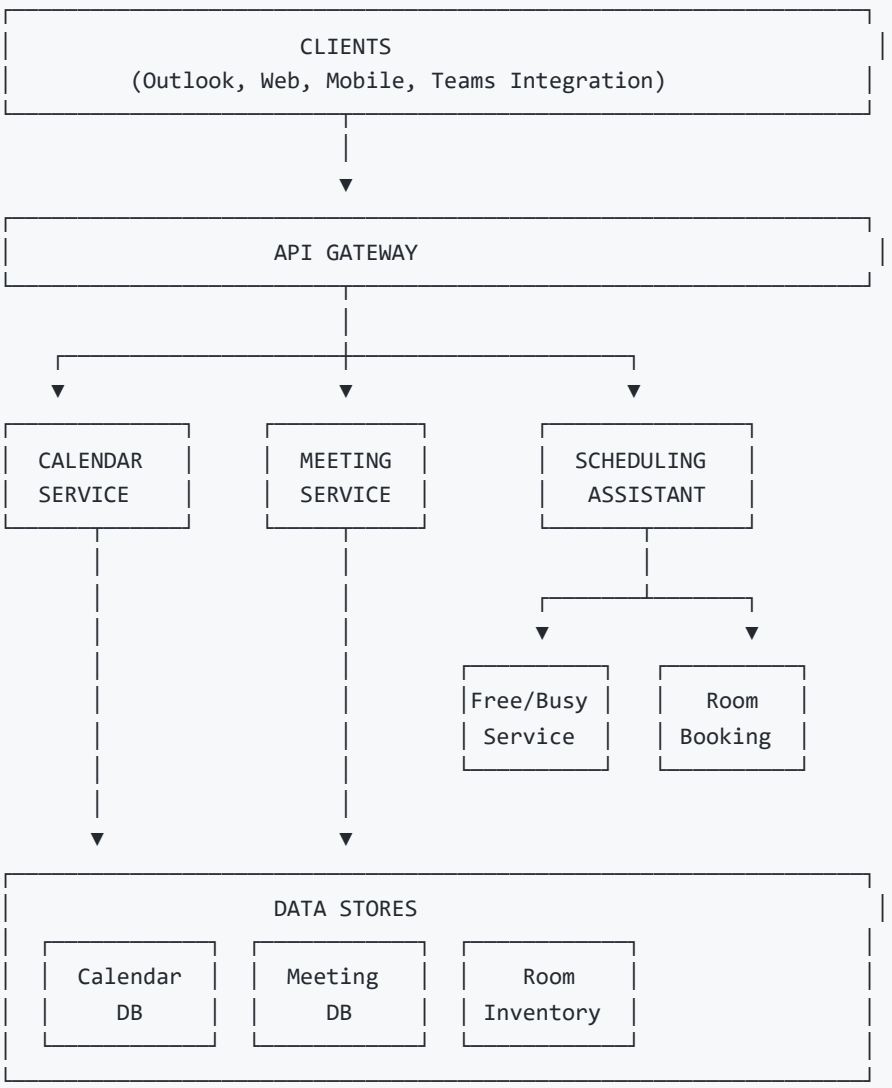
Design a calendar and meeting scheduling system like Outlook Calendar that handles time zones, recurring meetings, and room bookings.

Clarifying Questions to Ask

1. **Scale**: How many users? Meetings per day? (100M users, 50M meetings?)
2. **Features**: Free/busy, room booking, recurring meetings?
3. **Time zones**: Global users across all time zones?
4. **Integration**: Email invites, video conferencing links?
5. **Conflicts**: Auto-detect or allow double-booking?
6. **Availability**: Find available slots across multiple attendees?
7. **Reminders**: Email, push, SMS notifications?

High-Level Design





API Design

Meeting APIs

Method	Endpoint	Description
`POST`	`/api/v1/meetings`	Create/schedule a meeting
`GET`	`/api/v1/meetings/{meetingId}`	Get meeting details
`PUT`	`/api/v1/meetings/{meetingId}`	Update meeting
`DELETE`	`/api/v1/meetings/{meetingId}`	Cancel meeting
`POST`	`/api/v1/meetings/{meetingId}/rsvp`	RSVP to a meeting

Availability APIs

Method	Endpoint	Description
`GET`	`/api/v1/availability?userIds={ids}&date={date}`	Get free/busy slots
`POST`	`/api/v1/availability/suggest`	Suggest meeting times for participants

Schedule Meeting Request

```
POST /api/v1/meetings
{
  "title": "Design Review",
  "start_time": "2025-03-15T10:00:00Z",
  "end_time": "2025-03-15T11:00:00Z",
  "timezone": "America/Los_Angeles",
  "attendees": ["user_1", "user_2", "user_3"],
  "recurrence": {
    "pattern": "weekly",
    "days": ["monday", "wednesday"],
    "end_date": "2025-06-15"
  },
  "room_id": "room_42"
}
```

Calendar APIs

Method	Endpoint	Description
`GET`	`/api/v1/calendars/{userId}?start={date}&end={date}`	Get calendar events
`POST`	`/api/v1/calendars/{userId}/block`	Block time on calendar

Key Components

Component	Purpose	Technology
Calendar Service	CRUD for calendar events	PostgreSQL
Meeting Service	Meeting lifecycle	PostgreSQL
Free/Busy Service	Availability queries	Redis + Bitmap

Room Booking	Conference room management	PostgreSQL
Scheduling Assistant	Find optimal times	Algorithm service
Notification Service	Reminders, invites	Email, Push

Time Zone Handling

```
STORAGE: Always store in UTC
┌──────────────────────────────────────────────────────────────┐
│  Meeting: "Team standup"                                     │
│  Start: 2024-01-15T14:00:00Z  (UTC)                          │
│  End:   2024-01-15T14:30:00Z  (UTC)                          │
│  Timezone: America/New_York   (for display)                  │
│                                                              │
│  Display for NYC user: 9:00 AM - 9:30 AM EST                 │
│  Display for London user: 2:00 PM - 2:30 PM GMT              │
│  Display for Tokyo user: 11:00 PM - 11:30 PM JST             │
└──────────────────────────────────────────────────────────────┘

RECURRING MEETING EDGE CASES:
  "Every day at 9 AM" in NYC
  - During DST transition, 9 AM moves in UTC
  - Store: recurring rule + timezone
  - Expand: compute UTC times at query time
```

Recurring Meeting Rules (iCalendar RRULE)

```
EXAMPLES:
  FREQ=DAILY;COUNT=10
    → Every day for 10 occurrences

  FREQ=WEEKLY;BYDAY=MO,WE,FR;UNTIL=20241231
    → Every Mon, Wed, Fri until end of 2024

  FREQ=MONTHLY;BYMONTHDAY=15
    → 15th of every month

  FREQ=YEARLY;BYMONTH=1;BYMONTHDAY=1
    → Every January 1st

STORAGE OPTIONS:
  1. Store rule, expand on query (less storage, slower reads)
  2. Pre-expand instances (more storage, faster reads)

  Recommendation: Hybrid
    - Store rule for future expansion
    - Pre-expand next 6 months for fast queries
```

Free/Busy Algorithm

```
FINDING AVAILABLE SLOTS:
┌──────────────────────────────────────────────────────────────┐
│  Input: [Alice, Bob, Charlie], duration: 30min, range: this week│
│                                                              │
│  1. Fetch busy times for each attendee                       │
│     Alice:  [9-10, 14-15]                                    │
│     Bob:    [10-11, 14-16]                                   │
│     Charlie:[9-11]                                           │
│                                                              │
│  2. Merge into combined busy bitmap                          │
│     Combined: [9-11, 14-16]                                  │
│                                                              │
│  3. Find gaps ≥ 30 minutes                                   │
│     Available: [11-14, 16-EOD]                               │
│                                                              │
│  4. Rank by preference (working hours, fewer conflicts)      │
│     Suggestion: 11:00 AM - 11:30 AM                          │
└──────────────────────────────────────────────────────────────┘

BITMAP OPTIMIZATION:
  - Each day = 288 slots (5-minute granularity)
  - Store as bitmap: 288 bits = 36 bytes per user per day
  - Fast OR operation to combine multiple calendars
```

Database Schema

```sql
-- Events (includes meetings)
CREATE TABLE events (
    id BIGINT PRIMARY KEY,
    calendar_id BIGINT NOT NULL,
    title VARCHAR(255),
    description TEXT,
    start_time TIMESTAMPTZ NOT NULL,  -- UTC
    end_time TIMESTAMPTZ NOT NULL,    -- UTC
    timezone VARCHAR(50),             -- for display
    recurrence_rule TEXT,             -- RRULE format
    recurrence_id BIGINT,             -- parent for instances
    location VARCHAR(255),
    meeting_link TEXT,
    status VARCHAR(20),               -- confirmed, tentative, cancelled
    created_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ
);

-- Attendees
CREATE TABLE attendees (
    event_id BIGINT,
    user_id BIGINT,
```

```
    email VARCHAR(255),
    response_status VARCHAR(20),  -- accepted, declined, tentative
    is_organizer BOOLEAN,
    PRIMARY KEY (event_id, user_id)
);

-- Rooms
CREATE TABLE rooms (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    capacity INT,
    building VARCHAR(100),
    floor INT,
    amenities JSONB  -- {projector, whiteboard, video_conf}
);
```

Scalability Considerations

- **Read-heavy**: Cache user calendars in Redis
- **Partitioning**: Shard by user_id or calendar_id
- **Recurring expansion**: Pre-expand 6 months, lazy-expand beyond
- **Free/busy**: Bitmap indexes for fast availability queries
- **Notifications**: Async via message queue

Trade-offs

Decision	Option A	Option B	Recommendation
Recurrence	Store rule	Pre-expand	**Hybrid** - rule + 6mo expand
Time storage	Local time	UTC	**UTC** - consistent
Free/busy	Query events	Bitmap cache	**Bitmap** - faster
Conflicts	Prevent	Allow (warn)	**Allow + warn** - flexibility

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Bitmap-based free/busy caching for fast availability queries	Tech Guide
PostgreSQL	Primary storage for events, attendees, and room data	Tech Guide

How to Explain It Well

> **Opening**: "Calendar scheduling handles time zones, recurring events, and availability. I'll design for 100M users with fast availability queries."

> **Key talking points**:
>
> - "Store all times in UTC, convert to user timezone for display"
> - "RRULE format for recurring meetings, pre-expand 6 months"
> - "Bitmap indexes for O(1) free/busy lookups across multiple users"
> - "Room booking uses pessimistic locking to prevent double-booking"
> - "Handle DST transitions by storing timezone name, not offset"
